

Team SJA's Site

Radix Tree Autocomplete Engine

Database Seed Exporter • City Spelling Game • Live Benchmarks

■ 15,000 Cities

■ Sub-ms Lookups

■ Built-in Game

■ DB Exporter

■ Responsive

SJA



Autocomplete Engine

Instant prefix search
Custom Radix Tree
Interactive visualizer



Seed Exporter

JSON / SQL / CSV output
State & pattern filters
One-click download



Spelling Game

Scrambled city names
Progressive clues
Persistent scoring

What is Team SJA's Site?

A multi-purpose developer hub combining a search engine, a utility tool, and a game
— all powered by the same Radix Tree data structure.



Autocomplete Engine

Instant city prefix search
15,000 Indian cities loaded
Custom Radix Tree backend
Live SVG tree visualizer



Database Seed Exporter

Filter by state & suffix
JSON / SQL / CSV output
Auto schema generation
Copy or Download



Geoguess Spelling Game

Scrambled city challenges
Autocomplete hints
Clue reveal system
Persistent local scoring



Live Benchmarks

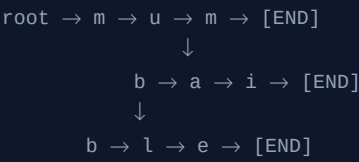
Real-time lookup speed
10,000-query stress test
Trie vs Radix comparison
ops/second throughput

Built with zero dependencies — pure HTML, CSS, and vanilla JavaScript.

What is a Radix Tree?

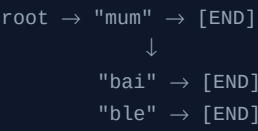
A Radix Tree compresses chains of single-child nodes into compact edge labels, dramatically reducing memory and improving cache performance.

Standard Trie — Uncompressed



■ 12 nodes for just 3 words
One character per node = wasteful

Radix Tree — Compressed ✓



■ 4 nodes – 70% fewer!
Edges store strings, not chars

O(L)

Search Complexity
(same as Trie)

~70%

Node Reduction
for Indian cities

3×

Memory Compaction
vs standard Trie

Edge Splitting Mechanics

When inserting a word, if a partial match occurs on an edge, the edge is split by introducing a branch node to hold the common prefix.

```
// 1. Find matching prefix length (j) on edge
let j = 0;
while (j < edge.length && edge[j] === word[i + j]) j++;

// 2. If partial match (j < edge.length), split!
const splitNode = new RadixNode();
const oldSuffix = edge.slice(j);
splitNode.children[oldSuffix[0]] = [oldSuffix, childNode];

// 3. Insert new suffix branch
const newSuffix = word.slice(i + j);
if (newSuffix) {
  const leafNode = new RadixNode();
  leafNode.isEnd = true;
  splitNode.children[newSuffix[0]] = [newSuffix, leafNode];
}

// 4. Update parent edge with shared prefix
parentNode.children[char] = [edge.slice(0, j), splitNode];
```

Layout & Audio Synthesis

Recursive coordinate plotting and runtime frequency synthesis keep the project lightweight, responsive, and dependency-free.



Trie Layout Mathematics

- Uses a post-order traversal to calculate nodes
- Leaf nodes are spaced evenly horizontally
- Parent nodes centered above children dynamically
- Bounding box maps seamlessly to SVG viewBox
- Scales to fit custom structures dynamically



Web Audio Sound Synthesis

- 100% browser-native sound generation
- Uses AudioContext & Oscillation Nodes
- Synthesizes arpeggios for correct guess chimes
- Low buzzer frequency synthesis for failures
- 0 KB external sound assets needed!

Layout Algorithm: $O(V + E)$ Recursive DFS
Audio Output: Real-time Sine & Triangle Waves

Autocomplete Engine

Type a prefix and instantly get matching Indian cities with sub-millisecond response.



Instant Search

15,000 city database
Sub-millisecond DFS
Top 10 suggestions
Prefix highlighting



Benchmarking

Live speed bar (5ms limit)
Exact microsecond timing
10,000-query stress test
ops/second throughput



Tree Visualizer

Drag & zoom SVG canvas
Playground: insert words
Search: trace query path
Animated edge splits

■ mum

■ mumbai, mumra, mumtaz... (0.34 ms)

Database Seed Exporter

Generate production-ready database seeding scripts for Indian city data in seconds.

1

Search & Add Cities

Use autocomplete to manually add specific cities to your seed basket.

2

Apply Filters

Filter by State (e.g. Maharashtra) or suffix (e.g. cities ending in 'pur', 'abad').

3

Choose Output Format

Pick JSON Array, PostgreSQL INSERT, MySQL INSERT, or CSV export.

4

Copy or Download

One-click copy to clipboard or download — ready to paste into your project.

```
INSERT INTO indian_cities_directory (city_name, zip_code, state_name) VALUES
('Mumbai', '400001', 'Maharashtra'),
('Delhi', '110001', 'Delhi'),
('Bengaluru', '560001', 'Karnataka');
```

Geoguess Spelling Game

An interactive game where players unscramble Indian city names for points and streaks.

SCRAMBLED CITY NAME

M U B I A M

Answer: MUMBAI



Scrambled Display

Random city name scrambled
6-character min length
No spaces or hyphens



Progressive Clues

Word length shown first
Third letter revealed
Last letter as final clue



Autocomplete Hints

Type partial guess
See matching city names
Uses the full 15K database



Scoring System

10 pts per correct answer
-2 pts per clue revealed
Streak tracked & saved

■ PERFORMANCE METRICS

By the Numbers

15,000

Indian Cities

<1ms

Avg Lookup

~70%

Node Reduction

500K+

Ops / Sec

0

Dependencies

Metric	Standard Trie	Radix Tree	Improvement
Total Nodes	~95,000	~28,000	70% fewer nodes
Memory Layout	Sparse char maps	Compressed edge strings	~3× compaction
Search Complexity	O(L) — one char/node	O(L) — one edge/node	Higher cache locality
Build Time	~180 ms	~220 ms (insert splits)	Negligible overhead
Prefix Query	DFS char traversal	DFS edge traversal	Fewer pointer hops

How the City Database Was Built

The 15,000-city dataset was prepared from the raw GeoNames geographical database.

1

Download & Extract

Fetches IN.zip from GeoNames containing all Indian geographical features in IN.txt.

2

Populated Place Filtering

Kept only Class P records (cities & towns). Removed lakes, monuments, admin regions.

3

Heuristic Sorting

Prioritized state & national capitals, then sorted by descending census population.

4

Clean & Deduplicate

Normalized to lowercase, collapsed spaces, removed punctuation, top 15,000 unique names.

Source: GeoNames.org • Feature Class P • Country Code: IN • Normalized UTF-8

[Source: geonames.org/export/](https://geonames.org/export/)

Open dataset • CC BY 4.0 License

TECH STACK

Built With Zero Dependencies

The entire site runs on pure browser-native technologies — no frameworks, no libraries.

- team-sja/
- index.html

– Structure: 3-tab layout, responsive viewport
- index.css

– Glassmorphism dark UI, animations, media queries
- app.js

– All logic: autocomplete, game, exporter, benchmark
- Web Audio, canvas particles, SVG renderer
- trie.js

– RadixTrie + StandardTrie implementations
- insert(), search(), autocomplete(), exportToJSON()
- cities.js

– 15,000 GeoNames Indian cities (208 KB)

- HTML5

Vanilla CSS

ES6+ JavaScript

Web Audio API

Canvas API

SVG DOM

localStorage

GitHub Pages
- Team SJA's Site • Radix Tree Autocomplete Engine & Developer Hub

iamhardikarora321-wq.github.io/team-sja

■ THANK YOU

Team SJA's Site

Built with ♥ using pure JavaScript — no frameworks, no dependencies,
just raw data structures and browser APIs.

■ iamhardikarora321-wq.github.io/team-sja

■ Team SJA

■ 15,000 Cities

■ Radix Tree

■ Spelling Game

■ DB Exporter

SJA